UTN FRLP

Desarrollo de Software 2026

React: Hooks y consumo de APIs

Clase #11 — 22 de junio de 2026

Matías Corti - Martina García Améndola

Temas de hoy

- Qué son los Hooks y por qué ya los venís usando
- useEffect: efectos secundarios y el array de dependencias
- Consumir datos con fetch, ahora dentro de React
- Estados de carga y error en la UI
- Tu primer custom hook: useFetch

De dónde venimos

La clase pasada vimos…

- **Formularios controlados** (value + onChange)
- **Validación con Zod** (safeParse, errores por campo)
- La lista de libros viviendo como **estado en App.tsx**

Hoy sumamos:

- Esos libros ya no nacen hardcodeados → **llegan de afuera**
- **useEffect** para pedirlos en el momento justo
- **useFetch** para no repetir el patrón en cada página



¿Qué son los Hooks?

¿Qué es un Hook?

Son funciones especiales que nos permiten "engancharnos" a las funcionalidades de React desde un componente.

Ya usamos tres sin saber qué eran Hooks:

Hook	Clase	Para qué
useState	C08	Guardar estado local
useNavigate	C09	Redirigir por código
useForm	C10	Manejar el formulario (RHF)

React.dev: Hooks



REGLAS IMPORTANTES

Reglas de los Hooks

1. Siempre en el nivel superior del componente

   → nunca dentro de if, loops o funciones anidadas

2. Solo en componentes o en otros custom hooks

¿Por qué?

React identifica cada Hook por el orden en que se llama, no por su nombre. Si el orden cambia entre renders, se rompe.



Efectos Secundarios

useEffect

¿Qué es?

El Hook para efectos secundarios: todo lo que tu componente hace además de dibujar la pantalla.

¿Por qué lo necesito?

React se encarga de mostrar. Pero pedir datos a una API, cambiar el título de la pestaña o armar un timer no es mostrar → eso va en `useEffect`.

```
useEffect(() => {
  // el efecto
}, [/* dependencias */]);
```

React.dev: useEffect



El array de dependencias

El segundo argumento decide cuándo se ejecuta el efecto:

Dependencia	Cuándo se ejecuta
Sin definir	Después de CADA render
`[]`	Una sola vez, al montar
`[libros]`	Al montar y cada vez que libros cambia

```
useEffect(() => {
  document.title = `Tenés ${libros.length} libros`;
}, [libros]); // ← se actualiza solo cuando cambia libros
```

⚡

Efectos Secundarios

useEffect

Ejemplo Básico

```tsx
import { useState, useEffect } from 'react';

function ContadorConTitulo() {
  const [contador, setContador] = useState<number>(0);

  // Solo se ejecuta cuando cambia 'contador'
  useEffect(() => {
    document.title = `Clicks: ${contador}`;
  }, [contador]); // <- Array de dependencias

  return (
    <button onClick={() => setContador(contador + 1)}>
      Clicks: {contador}
    </button>
  );
}

export default ContadorConTitulo;
```

🔁

¿Por qué corre dos veces?

StrictMode

En `main.tsx`, la app está envuelta en `<StrictMode>`.

¿Qué hace?

En desarrollo, monta cada componente dos veces a propósito, para detectar efectos mal escritos.

¿Por qué me importa?

Vas a ver tu console.log (y más tarde tu fetch) dos veces. **No es un bug**. En producción no pasa.

✅ Cierre Parte 1

Hasta acá:

- Un **Hook** es una función use… que te engancha a React
- **useEffect** = efectos secundarios
- El array decide cuándo:

  (sin array) | **[]** | **[deps]**

- **StrictMode** duplica en desarrollo (pero no en producción)



Fetch API



APIs

API = *Application Programming Interface*
(Interfaz de Programación de Aplicaciones)

¿Qué es una API?

Es una forma estandarizada que permite que dos sistemas se comuniquen entre sí.

Es como un **contrato** que define:

- Qué se puede pedir
- Cómo se debe pedir
- Qué se espera como respuesta

¿Qué cambia?

- Los datos ya no están "fijos" en el código.
- Se pueden obtener dinámicamente desde una base de datos.
- Se pueden actualizar, buscar, borrar o crear, en tiempo real.



APIs

Analogía clásica: Pedir comida 🍔

📱 **Aplicación** = Un *Cliente* en un restaurante

👨‍🍳 **Servidor** = La *Cocina*

🤵 **API** = El *Camarero*

¿Qué pasa?

1. El *cliente* (la app) **pide algo del menú.**
2. El *camarero* (la API) **lleva ese pedido a la cocina.**
3. La *cocina* (el servidor) **prepara el plato.**
4. El *camarero* **trae el plato al *cliente*.**

Lo importante:

- El *cliente* no entra a la *cocina*, ni *cocina* nada.
- Solo **hace un pedido claro** y **recibe una respuesta clara.**
- Eso es una API: **una manera de pedirle algo a otro sistema sin meterse en cómo lo hace.**



MOCK LOCAL

En caso que no tengamos una API, podemos emular eso poniendo los datos en **public/libros.json**

```
fetch('/libros.json')    // ← request HTTP real, se ve en la pestaña Network

// • Cero dependencias, funciona sin internet
// • Es el mismo mock que pide la Entrega 2 ("datos mock")
// • Las imágenes también van a public/

// Al hook le da igual la URL: '/libros.json' o 'https://api…'
// Mismo fetch.
```



APIs para Practicar

APIs con datos predefinidos

JSONPlaceholder https://jsonplaceholder.typicode.com
Casos: Lista de usuarios, Lista de posts, Lista de comentarios, etc

DummyJSON https://dummyjson.com/
Casos: Lista de productos, Lista de usuarios, Lista de citas, etc.

Beeceptor Mocks https://beeceptor.com/mock-server/explore/
Varios servicios

APIs para crear mocks personalizados

Beeceptor: https://beeceptor.com/

Mock-API.Net: https://mock-api.net

MockAPI: https://mockapi.io/

JsonServer.io: https://jsonserver.io/



FETCH LO VIMOS EN C05

En **C05** ya usaron fetch con TS plano (el buscador de Open Library, ¿se acuerdan?).

Lo nuevo no es fetch.

Lo nuevo es **cuándo dispararlo** dentro de un componente de React.

```
async function obtenerLibros(): Promise<LibroCardProps[]> {
  const res = await fetch('/libros.json');
  if (!res.ok) throw new Error('Error al cargar');
  return res.json();
}
```

¿DÓNDE VA EL FETCH?

❌ **NO VA en el cuerpo del componente:**

Cada render volvería a dispararlo → pide datos sin parar.

✅ **VA dentro de un useEffect con []:**

Se dispara una sola vez, al montar.

La receta (fetch es asincrónico):

```
useEffect(() => {
  const cargar = async () => {
    // await fetch(...)
  };
  cargar();          // definimos y llamamos
}, []);
```



El patrón de 3 estados

Toda vista que pide datos maneja tres estados:

```
const [libros, setLibros]   = useState<LibroCardProps[]>([]);
const [loading, setLoading] = useState(true);
const [error, setError]     = useState<string | null>(null);
```

Flujo:

loading: true
→ llega la respuesta
→ *setLibros(datos)*
→ *loading: false*
Si algo falla → *setError(...)*

Recuerden: *fetch NO rechaza con un 404. Hay que chequear res.ok a mano.*



La UI refleja el estado

Igual que en la clase anterior, si errors.titulo tenía contenido aparecía el feedback del campo, acá se mostrarán los componentes en base a los 3 estados

```
if (loading) return <Spinner animation="border" />;
if (error)   return <Alert variant="danger">{error}</Alert>;
return <Catalogo libros={libros} />;

// Spinner y Alert vienen de react-bootstrap (ya instalado).
// La UI es un reflejo del estado.
```



Fetch

Ejemplo Completo

```tsx
import { useState, useEffect } from 'react';
import { Spinner, Alert } from 'react-bootstrap';
import LibroCard from '../components/LibroCard';
import type { LibroCardProps } from '../types/libroCardProps';

function Catalogo() {
  const [libros, setLibros]   = useState<LibroCardProps[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError]     = useState<string | null>(null);

  useEffect(() => {
    const cargar = async () => {
      try {
        const res = await fetch('/libros.json');
        if (!res.ok) throw new Error('Error al cargar los libros');
        setLibros(await res.json());
      } catch (e) {
        setError(e instanceof Error ? e.message : 'Error desconocido');
      } finally {
        setLoading(false);
      }
    };
    cargar();
  }, []);

  if (loading) return <Spinner animation="border" />;
  if (error)   return <Alert variant="danger">{error}</Alert>;

  return (
    <div className="grid-libros">
      {libros.map(libro => <LibroCard key={libro.id} {...libro} />)}
    </div>
  );
}
```



App.tsx se reduce

ANTES: App guardaba la lista y la repartía por props

```
function App() {
  const [libros, setLibros] = useState<LibroCardProps[]>(librosIniciales);
  const agregarLibro = (nuevo: LibroCardProps) => setLibros([...libros,
nuevo]);

  return (
    <Layout>
      <Routes>
        <Route path="/catalogo"     element={<Catalogo libros={libros} />} />
        <Route path="/libros/nuevo" element={<LibroNuevo
                                              onAgregar={agregarLibro} />} />
      </Routes>
    </Layout>
 );
}
```

AHORA: Cada página pide lo suyo. App vuelve a ser solo el mapa de rutas

```
function App() {
  return (
    <Layout>
      <Routes>
        <Route path="/catalogo"     element={<Catalogo />} />
        <Route path="/libros/nuevo" element={<LibroNuevo />} />
      </Routes>
    </Layout>
  );
}
```



Leer vs. crear: el lado que falta

✅ **Leer** datos de afuera → lo resolvimos hoy con fetch.

❓ ¿**Y crear**? El formulario de C10 sigue ahí. Pero el catálogo ahora lee del JSON… tu libro nuevo no aparece.

¿Por qué? Guardar de verdad es mandar el dato a un servidor (un **POST**) que todavía no existe.

Lo construimos en el Bloque de Backend. Por ahora la maqueta nos sirve: tenemos el flujo de pantalla completo.

✅

Cierre Parte 2

Hasta acá:

- **API** = pedirle datos a otro sistema
- El `fetch` va en `useEffect([])`, con función `async` adentro
- Patrón de 3 estados:
  `datos` / `loading` / `error`
- La UI refleja el estado
  (`Spinner` / `Alert`)
- Crear datos necesita backend



Desafío: Encontrar el bug

~10 min

Usar el repositorio de referencia: DS26-Libreria

```
git pull
git checkout C11-bug
```

Iniciar la App e ingresar por el navegador.

La app empieza a pedir libros sin parar.

Abrí la pestaña Network y mirá.

🎯 **El reto:** diagnosticá qué está mal y arreglalo.

💡 **Pista:** mirá el **useEffect**... ¿qué le falta?



Custom Hooks



Custom Hooks

¿Qué es?

Son funciones que nos permiten extraer lógica compleja y reutilizarla. *Su nombre debe empezar con "use".*

¿Por qué?

- Reutilización: la misma lógica en muchos componentes
- Separación: la lógica afuera de la UI
- Testing: se prueba sola

```
function useToggle(inicial: boolean = false): [boolean, () => void] {
  const [valor, setValor] = useState<boolean>(inicial);
  const toggle = () => setValor(v => !v);
  return [valor, toggle];
}
```

[React.dev: Custom Hooks]



El problema: repetir el patrón

Ahora imaginen su TP:

- la home pide destacados
- el catálogo pide la lista
- el detalle pide un libro

Cada una repite las mismas ~15 líneas: los 3 estados, el useEffect, el try/catch, el res.ok

¿Y si lo escribimos una vez y lo reusamos?



El contrato de useFetch

Lo que recibe y lo que devuelve:

```
function useFetch<T>(url: string): {
  data: T | null;
  loading: boolean;
  error: string | null;
}

// El <T> es el mismo < > que ya usás en
useState<LibroCardProps[]>(...)
// le decimos al hook QUÉ TIPO de dato va a devolver.

const { data, loading, error } =
useFetch<LibroCardProps[]>('/libros.json');
```



Custom Hook

useFetch

```tsx
// frontend/src/hooks/useFetch.ts

import { useState, useEffect } from 'react';

export function useFetch<T>(url: string) {
  const [data, setData] = useState<T | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    const cargar = async () => {
      try {
        setLoading(true);
        setError(null);
        const res = await fetch(url);
        if (!res.ok) throw new Error('Error al cargar los datos');
        setData(await res.json());
      } catch (e) {
        setError(e instanceof Error ? e.message : 'Error desconocido');
      } finally {
        setLoading(false);
      }
    };
    cargar();
  }, [url]);

  return { data, loading, error };
}
```



Custom Hook

useFetch

Ejemplo de Uso

```tsx
// pages/Catalogo.tsx

import { Spinner, Alert } from 'react-bootstrap';
import LibroCard from '../components/LibroCard';
import { useFetch } from '../hooks/useFetch';
import type { LibroCardProps } from '../types/libroCardProps';

function Catalogo() {
  const { data: libros, loading, error } =
      useFetch<LibroCardProps[]>('/libros.json');
  if (loading) return <Spinner animation="border" />;
  if (error)   return <Alert variant="danger">{error}</Alert>;
  return (
    <div className="grid-libros">
      {(libros ?? []).map(libro => <LibroCard key={libro.id} {...libro}
/>)}
    </div>
  );
}

export default Catalogo;
```

✅

Cierre Parte 3

Hasta acá:

✅ Custom hook = lógica empaquetada y reusable

✅ useFetch<T> nos da los 3 estados en una línea

Actividades de la semana

Actividad Individual de la semana

1° instancia hasta el **28/06/2026**
*2° instancia: **05/07/2026** (Cierre Bloque 2)*

[AI-C11] Hooks en la librería

Incorporar Hooks y useFetch a la librería React

En el repositorio ds2026-actividades copiar el sitio de la librería de la Clase 10 en subdirectorio nuevo para esta clase: **ejercicios/c11-hooks/**

1. *Utilizar **useEffects** en alguna página o componente*
2. *El catálogo de tu librería tomando los datos de un **Mock** (public/*.json o una API) con **useFetch***
3. ***Spinner** mientras carga, **Alert** si falla*

Subir al repositorio y comunicar la finalización en Discord en un mensaje dentro del canal **#actividades**, en el Hilo: "***[AI-C11] Hooks en Librería***".

Actividad Grupal

Hasta el **28/06/2026** (inclusive)

RECORDATORIO
El **05/07/2026** es la **ENTREGA 2 (Frontend)** del **Trabajo Práctico Grupal**

[AG-C11] Mock + Hooks en el TP

Incorporar Hooks y useFetch en el TP Grupal

1. *Utilizar **useEffects** en alguna página o componente*
2. *Al menos un listado real del dominio del TP, consumiendo un **Mock** (public/*.json o una API mockeada)*
3. *Estados de loading y error visibles*
4. Dividir las tareas en el equipo
5. Hacer PRs cruzados (al menos uno cada alumno)
1. Trello: mover tarjetas correspondientes a "Done"
2. Comunicar la finalización en Discord en un mensaje único por equipo, dentro del canal **#actividades**, en el Hilo: "***[AG-C11] Mock + Hooks en el TP***".